WISP-MS Claude Master Prompt

Full-Stack Code Generation Prompt for Next.js 14 and Supabase Digitization System

Next.js 14 | App Router | Supabase | Tailwind | shadcn/ui | TypeScript

Prompt Usage

This document contains the complete production-ready prompt for generating the WiFi Service Provider Management System using Next.js 14 and Supabase.

- Use the prompt block below as-is in Claude, Cursor, or a compatible AI coding agent.
- The prompt covers database design, RLS policies, app architecture, UX requirements, and migration tooling.

Claude AI Master Prompt

COPYABLE PROMPT CODE *Target: Claude 3.5 Sonnet / Claude 3 Opus / Cursor*

```
# MASTER AI CODING PROMPT: NEXT-GENERATION WISP-MS

You are an expert Principal Full-Stack Engineer and Software Architect specializing in Next.js 14
(App Router), TypeScript, Tailwind CSS, shadcn/ui, and Supabase (PostgreSQL with Row Level
Security).

Your objective is to build a complete, fully functional, production-ready WiFi Service Provider
Management System (WISP-MS) based on the comprehensive system architecture document detailed below.
You must generate all necessary database migration scripts, Supabase schema, RLS policies, seed
data, TypeScript interfaces, Next.js App Router API routes, UI pages, and components.

---

## 1. PROJECT TECH STACK & REQUIREMENTS OVERVIEW

- Frontend & API Framework: Next.js 14 (App Router, Server Components, Server Actions, Route
Handlers)
- Language: TypeScript (Strict Mode)
- Styling & UI: Tailwind CSS, shadcn/ui components, Lucide Icons
- Backend & Database: Supabase (PostgreSQL 15+, Supabase Auth, Row Level Security, Realtime
Subscriptions)
- State Management & Data Fetching: React Query (TanStack Query v5) / Server Actions / Supabase
Client (@supabase/ssr)
- Validation: Zod schemas for all forms and API endpoints

---

## 2. DATABASE SCHEMA & SUPABASE SETUP (schema.sql)

Write a complete, single SQL file containing all PostgreSQL database migrations, extensions,
tables, foreign keys, triggers, enum types, indexes, and Row Level Security (RLS) policies.
```

Core Tables & Requirements:

1. packages (Internet Plans)
   - id: UUID DEFAULT gen_random_uuid() PRIMARY KEY
   - package_name: VARCHAR(50) NOT NULL UNIQUE (e.g., "Basic 10M", "Ultra 20M")
   - speed_mbps: INTEGER NOT NULL CHECK (speed_mbps > 0)
   - monthly_price: DECIMAL(10,2) NOT NULL CHECK (monthly_price >= 0)
   - created_at: TIMESTAMPTZ DEFAULT NOW()

2. customers (Client Directory)
   - id: UUID DEFAULT gen_random_uuid() PRIMARY KEY
   - full_name: VARCHAR(100) NOT NULL
   - phone_number: VARCHAR(20) NOT NULL UNIQUE
   - cnic_number: VARCHAR(20) UNIQUE
   - address: TEXT NOT NULL
   - package_id: UUID REFERENCES packages(id) ON DELETE SET NULL
   - status: VARCHAR(20) NOT NULL CHECK (status IN ('Active', 'Overdue', 'Suspended')) DEFAULT 'Active'
   - installation_date: DATE NOT NULL DEFAULT CURRENT_DATE
   - created_at: TIMESTAMPTZ DEFAULT NOW()
   - Add indexes on phone_number, full_name, and status for rapid searching across 50,000+ records.

3. payments (Financial Ledger)
   - id: UUID DEFAULT gen_random_uuid() PRIMARY KEY
   - customer_id: UUID NOT NULL REFERENCES customers(id) ON DELETE CASCADE
   - amount_paid: DECIMAL(10,2) NOT NULL CHECK (amount_paid > 0)
   - payment_date: DATE NOT NULL DEFAULT CURRENT_DATE
   - payment_month: VARCHAR(20) NOT NULL (e.g., "August 2026")
   - payment_method: VARCHAR(30) NOT NULL CHECK (payment_method IN ('Cash', 'EasyPaisa', 'JazzCash', 'Bank'))
   - received_by: VARCHAR(50) NOT NULL
   - created_at: TIMESTAMPTZ DEFAULT NOW()

4. complaints (Support Ticket Desk)
   - id: UUID DEFAULT gen_random_uuid() PRIMARY KEY
   - customer_id: UUID NOT NULL REFERENCES customers(id) ON DELETE CASCADE
   - issue_description: TEXT NOT NULL
   - priority: VARCHAR(20) NOT NULL CHECK (priority IN ('Low', 'Medium', 'High', 'Critical')) DEFAULT 'Medium'
   - status: VARCHAR(20) NOT NULL CHECK (status IN ('Open', 'In Progress', 'Resolved')) DEFAULT 'Open'
   - assigned_technician: VARCHAR(100)
   - resolution_notes: TEXT
   - created_at: TIMESTAMPTZ DEFAULT NOW()
   - updated_at: TIMESTAMPTZ DEFAULT NOW()

5. Row Level Security (RLS) & Triggers:
   - Enable RLS on all tables (packages, customers, payments, complaints).
   - Create access control policies for roles: Admin, Receptionist, Technician.
   - Add automatic updated_at triggers for tables with modification tracking.
   - Seed data script with default internet packages (10 Mbps, 20 Mbps, 50 Mbps) and mock customer records.

---

3. APPLICATION ARCHITECTURE & FOLDER STRUCTURE

Implement the following Next.js 14 App Router project structure:

```text
src/
├── app/
│   ├── (auth)/
│   │   ├── login/page.tsx
│   ├── (dashboard)/
│   │   ├── layout.tsx
│   │   ├── page.tsx                    # Global Search & Analytics Dashboard
│   │   ├── customers/
│   │   │   ├── page.tsx                # Customer Listing, Search & Filters
│   │   │   ├── new/page.tsx            # Customer Onboarding Form
│   │   │   └── [id]/page.tsx           # Customer Profile & Billing History
│   │   ├── packages/
│   │   │   └── page.tsx                # Package Management
│   │   ├── payments/
│   │   │   └── page.tsx                # Billing Ledger & Invoice Generation
│   │   ├── complaints/
│   │   │   └── page.tsx                # Support Desk & Technician Queue
│   │   └── migration/
│   │       └── page.tsx                # CSV Import Tool (Paper Log Digitization)
│   ├── api/
│   │   ├── customers/route.ts
│   │   ├── payments/route.ts
│   │   └── complaints/route.ts
│   └── layout.tsx
├── components/
│   ├── ui/                             # shadcn components (Button, Input, Table, Badge, Card,
Select, Dialog)
│   ├── dashboard/                      # Dashboard KPI cards and charts
│   ├── customers/                      # Customer forms & data tables
│   ├── payments/                       # Receipt printable view & billing form
│   ├── complaints/                     # Ticket status cards & assignment modal
│   └── shared/                         # Sidebar, Header, Global Search bar
├── lib/
│   ├── supabase/
│   │   ├── client.ts
│   │   ├── server.ts
│   │   └── middleware.ts
│   ├── utils.ts
│   └── validations.ts
└── types/
    └── database.ts                     # Generated Supabase TypeScript definitions
```

---

4. DETAILED FEATURE IMPLEMENTATION INSTRUCTIONS

Module 1: Executive Dashboard (/)

- KPI Summary Cards: Total Active Customers & Account Status Breakdown (Active, Overdue,
Suspended).
- Total Revenue Collected This Month vs. Pending Receivables.
- Open Technical Tickets (High/Critical alert badges).
- Global Search Bar: Instantly filter active database records by Name, Phone Number, CNIC, or

Address with live debounced search (< 200ms latency).
- Recent Activity Feed: Live updates on newly logged payments and resolved support tickets using Supabase Realtime subscriptions.

Module 2: Customer Onboarding & Profiling (/customers)

- FR-1 & FR-3 Implementation: Responsive Data Table with multi-column sorting, pagination, and status filters.
- Customer Creation Form (using React Hook Form + Zod validation) with fields: Full Name, Phone Number, CNIC, Installation Address, Package Selection (dropdown populated from packages), and Installation Date.
- Profile Detail View (/customers/[id]): Displays customer info, assigned package, subscription lifecycle status badge, full payment history ledger, open/closed complaint tickets, and quick actions ("Record Payment", "Raise Ticket", "Change Package", "Suspend Line").

Module 3: Package Management (/packages)

- FR-2 Implementation: CRUD interface for internet tiers (Name, Speed in Mbps, Monthly Cost).
- Shows subscriber count per package tier.

Module 4: Billing & Financial Ledger (/payments)

- FR-4 Implementation: Quick-payment logging form: Select Customer, Auto-fill Monthly Due Amount, Select Month, Payment Method (Cash, EasyPaisa, JazzCash, Bank), and Collector Name.
- Printable Digital Receipt Generator: A clean modal view formatting the receipt for instant printing or PDF download with business branding, transaction ID, customer details, and payment confirmation stamp.
- Revenue Breakdown Report: Tabular display of monthly collections vs delinquent balances.

Module 5: Support & Complaint Desk (/complaints)

- FR-5 Implementation: Ticket Management Board (Kanban / Table view) grouped by status (Open, In Progress, Resolved).
- Priority Tags: Low (Gray), Medium (Blue), High (Orange), Critical (Red).
- Ticket Creation Modal & Technician Assignment dropdown.
- Resolution notes input field upon closing tickets.

Module 6: Data Migration Tool (/migration)

- Section 6 SDLC Migration Pipe: CSV Bulk Upload Interface to digitize legacy paper registers.
- CSV Parser validating required headers (full_name, phone_number, cnic_number, address, package_name, status).
- Batch Insertion engine with error reporting and preview step before committing to PostgreSQL.

---

5. UI/UX & STYLING GUIDELINES

- Modern, clean SaaS dashboard aesthetic optimized for dual use: desktop administration and field technician mobile viewing.
- Dark/Light mode support using CSS variables.
- Clear status badges: Active (Emerald Green), Overdue (Amber/Orange), Suspended (Rose/Red).
- Optimized typography and accessible high-contrast UI elements.

---

6. EXPECTED OUTPUT

```
Generate the complete codebase structure including:

- Complete schema.sql (Tables, RLS policies, Indexes, Triggers, Seed Data).
- Supabase Server/Client utility helpers (lib/supabase/*).
- Zod validation schemas (lib/validations.ts).
- Next.js App Router Page components, Server Actions, and UI components for all modules.
- Python CSV Migration script (scripts/migrate_paper_logs.py) using supabase-py for automated bulk data ingestion as specified in Section 6.

---

### Final instruction to the AI agent

Build the complete system described above inside a new Next.js 14 project. Ensure the database schema, API layer, UI modules, and migration tooling all align with the specifications. Prefer production-quality code and real-world best practices. Include clear comments where needed, but do not over-explain trivial logic. Output only the final implementation code and required files, with no placeholder text.

---

## Additional Notes for the Implementer

- Use proper environment variable handling for Supabase connection strings and service roles.
- Never expose secret keys in frontend code.
- Ensure all forms use validation schemas and server-side checks.
- Include a seeded default package catalog and sample customer records.
- Provide a migration utility that accepts CSV input while keeping the system robust against malformed rows.
- Use accessible components and mobile-responsive layouts.
- Make sure the dashboard is suitable for both admin operators and technicians performing field checks.

---

## MASTER PROMPT END
```